

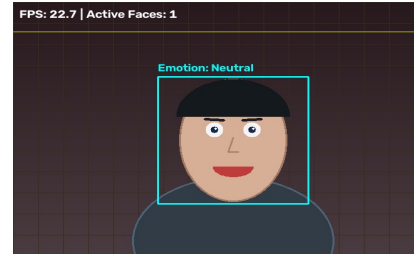
01. Task 01: Real-Time Face Detection

File: `task01_face_detection.py` | Total Lines: 84

```

01 """
02 Task 01: Real-Time Face Detection
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Processes live frames and detects faces using OpenCV Haar Cascades in a real-time while loop.
05 """
06
07 import cv2
08 import time
09 import os
10 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
11
12 def run_face_detection(camera_idx=0, save_output=True):
13     # Load Haar cascade classifier
14     cascade_path = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
15     face_cascade = cv2.CascadeClassifier(cascade_path)
16
17     # Capture real-time video input from default webcam
18     cap = open_webcam(camera_idx)
19     if not cap.isOpened():
20         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
21         return
22
23     output_dir = ensure_output_dir()
24     saved_sample = False
25     prev_time = time.time()
26
27     print("\n--- [TASK 01: REAL-TIME FACE DETECTION] ---")
28     print("Capturing live video from webcam...")
29     print("Press 'q' or 'ESC' on display window to exit.\n")
30
31     window_name = "Task 01 - Real-Time Face Detection"
32
33     # Real-time processing loop
34     while True:
35         ret, frame = cap.read()
36         if not ret or frame is None:
37             print("[ERROR] Failed to read live frame from webcam.")
38             break
39
40         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
41
42         # Real-time multi-scale face detection
43         faces = face_cascade.detectMultiScale(
44             gray,
45             scaleFactor=1.1,
46             minNeighbors=5,
47             minSize=(30, 30)
48         )
49
50         # Draw bounding boxes around detected live faces
51         for (x, y, w, h) in faces:
52             cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
53             cv2.putText(frame, "Face", (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
54
55         # Calculate live FPS
56         curr_time = time.time()
57         fps = 1.0 / (curr_time - prev_time + 1e-5)
58         prev_time = curr_time
59
60         # Display info banner & tech HUD
61         cv2.putText(frame, f"Live Faces Detected: {len(faces)} | FPS: {fps:.1f}", (15, 30),
62                     cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
63         draw_tech_hud_grid(frame)
64
65         # Save single output snapshot if requested
66         if save_output and not saved_sample and len(faces) > 0:
67             out_path = os.path.join(output_dir, "task01_face_detection_result.jpg")
68             cv2.imwrite(out_path, frame)
69             print(f"[SUCCESS] Saved live detection result snapshot to: {out_path}")
70             saved_sample = True
71
72         # Display real-time output
73         cv2.imshow(window_name, frame)
74
75         key = cv2.waitKey(1) & 0xFF
76         if key == ord('q') or key == 27:
77             break
78
79     cap.release()
80     cv2.destroyAllWindows()
81     print("[TASK 01 COMPLETED]")
82
83 if __name__ == "__main__":
84     run_face_detection()

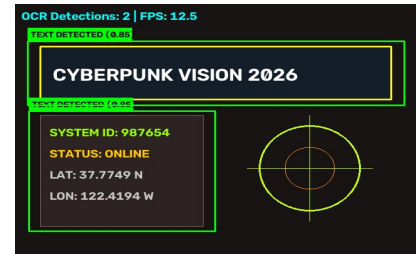
```



02. Task 02: Facial Expression & Emotion Recognition

File: task02_expression.py | Total Lines: 108

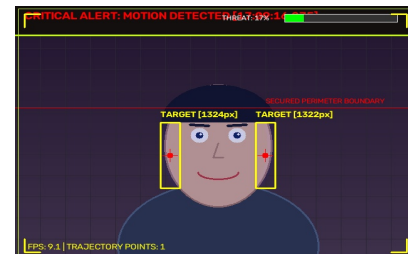
```
01 """
02 Task 02: Real-Time Facial Expression & Emotion Recognition
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Processes live frames and classifies facial expressions in a real-time while loop.
05 """
06
07 import cv2
08 import time
09 import os
10 import numpy as np
11 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
12
13 def detect_expression(frame, face_cascade, smile_cascade, eye_cascade):
14     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
15     faces = face_cascade.detectMultiScale(gray, 1.1, 5, minSize=(60, 60))
16
17     for (x, y, w, h) in faces:
18         roi_gray = gray[y:y+h, x:x+w]
19         roi_color = frame[y:y+h, x:x+w]
20
21         # Detect smile within face ROI
22         smiles = smile_cascade.detectMultiScale(roi_gray, scaleFactor=1.7, minNeighbors=20, minSize=(25, 25))
23         # Detect eyes within upper face ROI
24         eyes = eye_cascade.detectMultiScale(roi_gray[0:int(h*0.6), :], scaleFactor=1.1, minNeighbors=5)
25
26         # Expression Classification Heuristics
27         expression = "Neutral"
28         color = (255, 255, 0)
29
30         if len(smiles) > 0:
31             expression = "Happy / Smiling"
32             color = (0, 255, 0)
33         elif len(eyes) >= 2:
34             mouth_roi = roi_gray[int(h*0.65):h, int(w*0.2):int(w*0.8)]
35             if mouth_roi.size > 0:
36                 thresh = cv2.threshold(mouth_roi, 60, 255, cv2.THRESH_BINARY_INV)
37                 dark_pixels = cv2.countNonZero(thresh)
38                 ratio = dark_pixels / (mouth_roi.shape[0] * mouth_roi.shape[1] + 1e-5)
39                 if ratio > 0.4:
40                     expression = "Surprised / Open Mouth"
41                     color = (0, 165, 255)
42
43         # Draw Face Bounding Box & Label
44         cv2.rectangle(frame, (x, y), (x+w, y+h), color, 2)
45         cv2.putText(frame, f"Emotion: {expression}", (x, y - 10),
46                     cv2.FONT_HERSHEY_SIMPLEX, 0.65, color, 2)
47
48         # Draw Eye boxes
49         for (ex, ey, ew, eh) in eyes:
50             cv2.rectangle(roi_color, (ex, ey), (ex+ew, ey+eh), (255, 0, 0), 1)
51
52         # Draw Smile boxes
53         for (sx, sy, sw, sh) in smiles:
54             cv2.rectangle(roi_color, (sx, sy), (sx+sw, sy+sh), (0, 255, 255), 1)
55
56     return frame, len(faces)
57
58 def run_expression_detection(camera_idx=0, save_output=True):
59     face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
60     smile_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_smile.xml')
61     eye_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_eye.xml')
62
63     cap = open_webcam(camera_idx)
64     if not cap.isOpened():
65         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
66         return
67
68     output_dir = ensure_output_dir()
69     saved_sample = False
70     prev_time = time.time()
71     window_name = "Task 02 - Facial Expression Recognition"
72
73     print("\n--- [TASK 02: FACIAL EXPRESSION RECOGNITION] ---")
74     print("Capturing live video from webcam...")
75     print("Press 'q' or 'ESC' on display window to exit.\n")
76
77     while True:
78         ret, frame = cap.read()
79         if not ret or frame is None:
80             break
81
82         processed_frame, num_faces = detect_expression(frame, face_cascade, smile_cascade, eye_cascade)
83
84         curr_time = time.time()
85         fps = 1.0 / (curr_time - prev_time + 1e-5)
86         prev_time = curr_time
87
88         cv2.putText(processed_frame, f"FPS: {fps:.1f} | Active Faces: {num_faces}", (15, 30),
89                     cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
90         draw_tech_hud_grid(processed_frame)
91
92         if save_output and not saved_sample and num_faces > 0:
93             out_path = os.path.join(output_dir, "task02_expression_result.jpg")
94             cv2.imwrite(out_path, processed_frame)
95             print(f"[SUCCESS] Saved expression result image to: {out_path}")
96             saved_sample = True
97
98         cv2.imshow(window_name, processed_frame)
99         key = cv2.waitKey(1) & 0xFF
100         if key == ord('q') or key == 27:
101             break
102
103     cap.release()
104     cv2.destroyAllWindows()
105     print("[TASK 02 COMPLETED]")
106
107 if __name__ == "__main__":
108     run_expression_detection()
```



03. Task 03: Optical Character Recognition (OCR)

File: task03_ocr.py | Total Lines: 115

```
01 """
02 Task 03: Real-Time Optical Character Recognition (OCR)
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Detects and reads text from live video frames using OpenCV text contour analysis & EasyOCR.
05 """
06
07 import cv2
08 import time
09 import os
10 import numpy as np
11 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
12
13 easyocr_reader = None
14
15 def get_easyocr_reader():
16     global easyocr_reader
17     if easyocr_reader is None:
18         try:
19             import easyocr
20             easyocr_reader = easyocr.Reader(['en'], gpu=False)
21             print("[INFO] EasyOCR initialized successfully.")
22         except Exception as e:
23             print(f"[NOTICE] EasyOCR not loaded ({e}). Using real-time OpenCV text contour detection.")
24             easyocr_reader = False
25     return easyocr_reader
26
27 def perform_opencv_contour_ocr(frame):
28     """Real-time text detection using adaptive thresholding and contour bounding boxes."""
29     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
30     blur = cv2.GaussianBlur(gray, (5, 5), 0)
31     thresh = cv2.adaptiveThreshold(blur, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
32                                   cv2.THRESH_BINARY_INV, 11, 2)
33
34     kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (15, 5))
35     dilated = cv2.dilate(thresh, kernel, iterations=2)
36
37     contours, _ = cv2.findContours(dilated, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
38     results = []
39
40     for cnt in contours:
41         x, y, w, h = cv2.boundingRect(cnt)
42         aspect_ratio = w / float(h)
43         area = w * h
44
45         if area > 400 and aspect_ratio > 1.2 and w > 40:
46             results.append([(x, y), (x+w, y), (x+w, y+h), (x, y+h)], "TEXT DETECTED", 0.85))
47
48     return results
49
50 def run_ocr(camera_idx=0, save_output=True):
51     cap = open_webcam(camera_idx)
52     if not cap.isOpened():
53         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
54         return
55
56     output_dir = ensure_output_dir()
57     reader = get_easyocr_reader()
58     saved_sample = False
59     prev_time = time.time()
60     window_name = "Task 03 - Real-Time Optical Character Recognition"
61
62     print("\n--- [TASK 03: REAL-TIME OCR] ---")
63     print("Capturing live video from webcam... Hold up any text to the camera!")
64     print("Press 'q' or 'ESC' on display window to exit.\n")
65
66     while True:
67         ret, frame = cap.read()
68         if not ret or frame is None:
69             break
70
71         annotated_frame = frame.copy()
72
73         if reader:
74             ocr_results = reader.readtext(frame)
75         else:
76             ocr_results = perform_opencv_contour_ocr(frame)
77
78         text_count = 0
79         for bbox, text, prob in ocr_results:
80             if prob > 0.3:
81                 text_count += 1
82                 pts = np.array(bbox, np.int32).reshape((-1, 1, 2))
83                 cv2.polylines(annotated_frame, [pts], True, (0, 255, 0), 2)
84
85                 top_left = (int(bbox[0][0]), int(bbox[0][1]))
86                 cv2.rectangle(annotated_frame, (top_left[0], top_left[1] - 25),
87                               (top_left[0] + len(text)*12 + 10, top_left[1]), (0, 255, 0), -1)
88                 cv2.putText(annotated_frame, f"{text} ({prob:.2f})", (top_left[0] + 5, top_left[1] - 7),
89                             cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 2)
90
91         curr_time = time.time()
92         fps = 1.0 / (curr_time - prev_time + 1e-5)
93         prev_time = curr_time
94
95         cv2.putText(annotated_frame, f"OCR Live Detections: {text_count} | FPS: {fps:.1f}", (15, 30),
96                     cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
97         draw_tech_hud_grid(annotated_frame)
98
99         if save_output and not saved_sample and text_count > 0:
100             out_path = os.path.join(output_dir, "task03_ocr_result.jpg")
101             cv2.imwrite(out_path, annotated_frame)
102             print(f"[SUCCESS] Saved OCR result image to: {out_path}")
103             saved_sample = True
104
105         cv2.imshow(window_name, annotated_frame)
106         key = cv2.waitKey(1) & 0xFF
107         if key == ord('q') or key == 27:
108             break
109
110     cap.release()
111     cv2.destroyAllWindows()
112     print("[TASK 03 COMPLETED]")
113
114 if __name__ == "__main__":
115     run_ocr()
```



04. Task 04: Real-Time Motion Security Intelligence

File: task04_motion_alert.py | Total Lines: 137

```

01 """
02 Task 04: Real-Time Motion Alert & Security Intelligence System
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Processes live frames for motion detection, tracking, and security alerts in a while True loop.
05 """
06
07 import cv2
08 import time
09 import os
10 import datetime
11 import numpy as np
12 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
13
14 def run_motion_alert(camera_idx=0, save_output=True, min_area=600):
15     cap = open_webcam(camera_idx)
16     if not cap.isOpened():
17         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
18         return
19
20     output_dir = ensure_output_dir()
21     first_frame = None
22     saved_sample = False
23     prev_time = time.time()
24     motion_event_count = 0
25     motion_history = []
26     heatmap = None
27     window_name = "Task 04 - Real-Time Motion Security Intelligence"
28
29     print("\n--- [TASK 04: REAL-TIME MOTION SECURITY INTELLIGENCE] ---")
30     print("Capturing live video from webcam...")
31     print("Press 'q' or 'ESC' on display window to exit.\n")
32
33     while True:
34         ret, frame = cap.read()
35         if not ret or frame is None:
36             break
37
38         h, w, c = frame.shape
39         if heatmap is None or heatmap.shape[2] != (h, w):
40             heatmap = np.zeros((h, w), dtype=np.float32)
41
42         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
43         blur = cv2.GaussianBlur(gray, (21, 21), 0)
44
45         if first_frame is None:
46             first_frame = blur
47             continue
48
49         # Frame Differencing
50         frame_delta = cv2.absdiff(first_frame, blur)
51         thresh = cv2.threshold(frame_delta, 22, 255, cv2.THRESH_BINARY)[1]
52         thresh = cv2.dilate(thresh, None, iterations=3)
53
54         # Accumulate motion heatmap
55         heatmap = cv2.addWeighted(heatmap, 0.94, thresh.astype(np.float32) / 255.0, 0.06, 0)
56
57         contours, _ = cv2.findContours(thresh.copy(), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
58         motion_detected = False
59         active_centroids = []
60         annotated = frame.copy()
61
62         # Render Motion Heatmap tint overlay
63         heatmap_norm = np.clip(heatmap * 255, 0, 255).astype(np.uint8)
64         heatmap_color = cv2.applyColorMap(heatmap_norm, cv2.COLORMAP_JET)
65         annotated = cv2.addWeighted(annotated, 0.82, heatmap_color, 0.18, 0)
66
67         total_moving_area = 0
68         for c_contour in contours:
69             area = cv2.contourArea(c_contour)
70             if area < min_area:
71                 continue
72
73             total_moving_area += area
74             (x, y, bw, bh) = cv2.boundingRect(c_contour)
75             cx, cy = x + bw // 2, y + bh // 2
76             active_centroids.append((cx, cy))
77
78         # Target Box & Reticle
79         cv2.rectangle(annotated, (x, y), (x + bw, y + bh), (0, 255, 255), 2)
80         cv2.circle(annotated, (cx, cy), 5, (0, 0, 255), -1)
81         cv2.line(annotated, (cx - 12, cy), (cx + 12, cy), (0, 0, 255), 1)
82         cv2.line(annotated, (cx, cy - 12), (cx, cy + 12), (0, 0, 255), 1)
83
84         cv2.putText(annotated, f"MOTION [{area:.0f}px]", (x, y - 10),
85                   cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 2)
86         motion_detected = True
87
88         if active_centroids:
89             motion_history.append(active_centroids[0])
90             if len(motion_history) > 30:
91                 motion_history.pop(0)
92
93         for i in range(1, len(motion_history)):
94             cv2.line(annotated, motion_history[i-1], motion_history[i], (0, 255, 255), 2)
95
96         threat_level = min(100, int((total_moving_area / (w * h * 0.05)) * 100))
97         cv2.rectangle(annotated, (0, 0), (w, 45), (15, 15, 20), -1)
98         cv2.line(annotated, (0, 45), (w, 45), (0, 255, 200), 2)
99
100         curr_time_str = datetime.datetime.now().strftime("%H:%M:%S")
101
102         if motion_detected:
103             motion_event_count += 1
104             cv2.putText(annotated, f"LIVE MOTION ALERT [{curr_time_str}] | THREAT: {threat_level}%", (15, 28),
105                       cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 0, 255), 2)
106         else:
107             cv2.putText(annotated, f"SYSTEM MONITORING - ALL CLEAR [{curr_time_str}]", (15, 28),
108                       cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 0), 2)
109
110         draw_tech_hud_grid(annotated)
111
112         # Periodically update reference frame
113         first_frame = cv2.addWeighted(first_frame, 0.95, blur, 0.05, 0)
114
115         curr_time = time.time()
116         fps = 1.0 / (curr_time - prev_time + 1e-5)
117         prev_time = curr_time
118         cv2.putText(annotated, f"FPS: {fps:.1f}", (20, h - 20),
119                   cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 1)
120
121         if save_output and not saved_sample and motion_detected:
122             out_path = os.path.join(output_dir, "task04_motion_alert_result.jpg")
123             cv2.imwrite(out_path, annotated)
124             print(f"[SUCCESS] Saved motion alert snapshot to: {out_path}")
125             saved_sample = True
126
127         cv2.imshow(window_name, annotated)
128         key = cv2.waitKey(1) & 0xFF
129         if key == ord('q') or key == 27:
130             break
131

```

```
132     cap.release()
133     cv2.destroyAllWindows()
134     print(f"[TASK 04 COMPLETED] Total Motion Events Detected: {motion_event_count}")
135
136 if __name__ == "__main__":
137     run_motion_alert()
```



05. Task 05: Virtual Air Canvas / Air Drawing

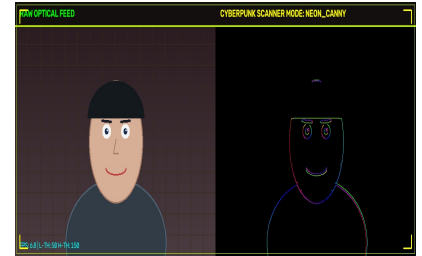
File: task05_drawing.py | Total Lines: 140

```
01 """
02 Task 05: Real-Time Virtual Air Canvas / Air Drawing
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Draws on live camera feed using MediaPipe hand index finger tracking or color pointer in a while True loop.
05 """
06
07 import cv2
08 import time
09 import os
10 import numpy as np
11 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
12
13 def apply_glow_effect(canvas):
14     """Applies a smooth neon glow effect to canvas strokes."""
15     glow = cv2.GaussianBlur(canvas, (21, 21), 0)
16     return cv2.addWeighted(canvas, 1.0, glow, 0.8, 0)
17
18 def run_air_drawing(camera_idx=0, save_output=True):
19     cap = open_webcam(camera_idx)
20     if not cap.isOpened():
21         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
22         return
23
24     output_dir = ensure_output_dir()
25     mp_hands = None
26     try:
27         import mediapipe as mp
28         if hasattr(mp, 'solutions') and hasattr(mp.solutions, 'hands'):
29             mp_hands = mp.solutions.hands.Hands(
30                 max_num_hands=1,
31                 min_detection_confidence=0.6,
32                 min_tracking_confidence=0.6
33             )
34             print("[INFO] MediaPipe Hand Tracking active.")
35     except Exception as e:
36         print(f"[NOTICE] Hand Tracking unavailable ({e}).")
37
38     canvas = None
39     colors = [(255, 255, 0), (0, 255, 120), (255, 0, 200), (0, 200, 255), (0, 0, 0)]
40     color_names = ["CYAN", "GREEN", "MAGENTA", "GOLD", "ERASER"]
41     current_color_idx = 0
42     brush_thickness = 8
43     prev_point = None
44     saved_sample = False
45     window_name = "Task 05 - Real-Time Virtual Air Canvas"
46
47     print("\n--- [TASK 05: REAL-TIME VIRTUAL AIR CANVAS] ---")
48     print("Capturing live video from webcam...")
49     print("Point your index finger in front of camera to draw!")
50     print("Press 'c' to Clear Canvas, 'q' or 'ESC' to exit.\n")
51
52     while True:
53         ret, frame = cap.read()
54         if not ret or frame is None:
55             break
56
57         frame = cv2.flip(frame, 1)
58         h, w, c = frame.shape
59
60         if canvas is None:
61             canvas = np.zeros((h, w, 3), dtype=np.uint8)
62
63         # Top Palette Control Toolbar
64         toolbar_h = 60
65         btn_w = w // len(colors)
66         cv2.rectangle(frame, (0, 0), (w, toolbar_h), (20, 20, 25), -1)
67         cv2.line(frame, (0, toolbar_h), (w, toolbar_h), (0, 255, 200), 2)
68
69         for i, col in enumerate(colors):
70             x1, y1 = i * btn_w, 0
71             x2, y2 = (i + 1) * btn_w, toolbar_h
72             is_selected = (i == current_color_idx)
73
74             bg_col = (40, 40, 40) if i == len(colors)-1 else tuple([int(c*0.5) for c in col])
75             cv2.rectangle(frame, (x1 + 4, 6), (x2 - 4, toolbar_h - 6), bg_col, -1)
76
77             border_col = (0, 255, 255) if is_selected else (80, 80, 80)
78             cv2.rectangle(frame, (x1 + 4, 6), (x2 - 4, toolbar_h - 6), border_col, 3 if is_selected else 1)
79             cv2.putText(frame, color_names[i], (x1 + 15, 30),
80                         cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 255, 255), 2 if is_selected else 1)
81
82         draw_point = None
83         if mp_hands:
84             rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
85             results = mp_hands.process(rgb_frame)
86
87             if results.multi_hand_landmarks:
88                 for hand_landmarks in results.multi_hand_landmarks:
89                     index_tip = hand_landmarks.landmark[8]
90                     cx, cy = int(index_tip.x * w), int(index_tip.y * h)
91                     draw_point = (cx, cy)
92                     cv2.circle(frame, draw_point, 12, colors[current_color_idx], -1)
93                     cv2.circle(frame, draw_point, 16, (255, 255, 255), 2)
94
95         if draw_point is not None:
96             if draw_point[1] < toolbar_h:
97                 selected_idx = draw_point[0] // btn_w
98                 if 0 <= selected_idx < len(colors):
99                     current_color_idx = selected_idx
100                     prev_point = None
101             else:
102                 if prev_point is not None:
103                     thickness = 30 if current_color_idx == 4 else brush_thickness
104                     cv2.line(canvas, prev_point, draw_point, colors[current_color_idx], thickness)
105                     prev_point = draw_point
106             else:
107                 prev_point = None
108
109         glow_canvas = apply_glow_effect(canvas)
110         gray_c = cv2.cvtColor(glow_canvas, cv2.COLOR_BGR2GRAY)
111         mask = cv2.threshold(gray_c, 5, 255, cv2.THRESH_BINARY_INV)
112         mask = cv2.cvtColor(mask, cv2.COLOR_GRAY2BGR)
113
114         combined = cv2.bitwise_and(frame, mask)
115         combined = cv2.add(combined, glow_canvas)
116         draw_tech_hud_grid(combined)
117
118         cv2.putText(combined, f"BRUSH: {color_names[current_color_idx]}", (20, h - 20),
119                     cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 1)
120
121         if save_output and not saved_sample and np.sum(canvas) > 0:
122             out_path = os.path.join(output_dir, "task05_drawing_result.jpg")
123             cv2.imwrite(out_path, combined)
124             print(f"[SUCCESS] Saved drawing result to: {out_path}")
125             saved_sample = True
126
127         cv2.imshow(window_name, combined)
128         key = cv2.waitKey(1) & 0xFF
129         if key == ord('c'):
130             canvas = np.zeros((h, w, 3), dtype=np.uint8)
131             print("[INFO] Canvas Cleared.")
```

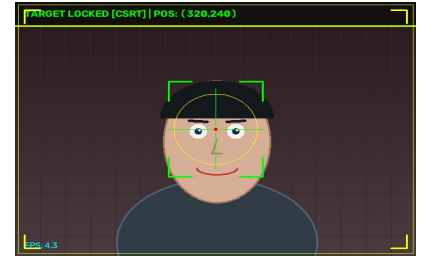
```
132         elif key == ord('q') or key == 27:  
133             break  
134  
135         cap.release()  
136         cv2.destroyAllWindows()  
137         print("[TASK 05 COMPLETED]")  
138  
139     if __name__ == "__main__":  
140         run_air_drawing()
```

06. Task 06: Interactive Real-Time Edge Scanner

File: task06_edge_detection.py | Total Lines: 125



```
01 """
02 Task 06: Real-Time Interactive Edge Detection & Scanner
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Processes live frames for Canny/Sobel/Laplacian edge filtering in a real-time while loop.
05 """
06
07 import cv2
08 import time
09 import os
10 import numpy as np
11 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
12
13 def nothing(x):
14     pass
15
16 def apply_cyberpunk_edge(frame, method="neon_canny", low_thresh=50, high_thresh=150):
17     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
18     blur = cv2.GaussianBlur(gray, (5, 5), 0)
19
20     if method == "neon_canny":
21         edges = cv2.Canny(blur, low_thresh, high_thresh)
22         gx = cv2.Sobel(blur, cv2.CV_32F, 1, 0, ksize=3)
23         gy = cv2.Sobel(blur, cv2.CV_32F, 0, 1, ksize=3)
24         _, angle = cv2.cartToPolar(gx, gy, angleInDegrees=True)
25
26         hsv = np.zeros((frame.shape[0], frame.shape[1], 3), dtype=np.uint8)
27         hsv[..., 0] = (angle / 2).astype(np.uint8)
28         hsv[..., 1] = 255
29         hsv[..., 2] = np.where(edges > 0, 255, 0).astype(np.uint8)
30         return cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
31
32     elif method == "thermal_sobel":
33         sobelx = cv2.Sobel(blur, cv2.CV_64F, 1, 0, ksize=3)
34         sobely = cv2.Sobel(blur, cv2.CV_64F, 0, 1, ksize=3)
35         mag = cv2.magnitude(sobelx, sobely)
36         mag_norm = np.clip(mag / (mag.max() + 1e-5) * 255, 0, 255).astype(np.uint8)
37         return cv2.applyColorMap(mag_norm, cv2.COLORMAP_INFERNO)
38
39     elif method == "laplacian_matrix":
40         lap = cv2.Laplacian(blur, cv2.CV_64F)
41         lap_abs = cv2.convertScaleAbs(lap)
42         return cv2.applyColorMap(lap_abs, cv2.COLORMAP_OCEAN)
43
44     else:
45         edges = cv2.Canny(blur, low_thresh, high_thresh)
46         return cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)
47
48 def run_edge_detection(camera_idx=0, save_output=True):
49     cap = open_webcam(camera_idx)
50     if not cap.isOpened():
51         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
52         return
53
54     output_dir = ensure_output_dir()
55     window_name = "Task 06 - Real-Time Edge Scanner"
56
57     try:
58         cv2.namedWindow(window_name, cv2.WINDOW_NORMAL)
59         cv2.createTrackbar("Low Thresh", window_name, 50, 255, nothing)
60         cv2.createTrackbar("High Thresh", window_name, 150, 255, nothing)
61     except cv2.error:
62         pass
63
64     current_mode = "neon_canny"
65     saved_sample = False
66     prev_time = time.time()
67
68     print("\n-- [TASK 06: REAL-TIME EDGE SCANNER] ---")
69     print("Capturing live video from webcam...")
70     print("Keyboard Controls: '1' -> Neon Canny | '2' -> Sobel | '3' -> Laplacian | 'q'/'ESC' -> Exit\n")
71
72     while True:
73         ret, frame = cap.read()
74         if not ret or frame is None:
75             break
76
77         low_val, high_val = 50, 150
78         try:
79             low_val = cv2.getTrackbarPos("Low Thresh", window_name)
80             high_val = cv2.getTrackbarPos("High Thresh", window_name)
81         except cv2.error:
82             pass
83
84         edge_map = apply_cyberpunk_edge(frame, method=current_mode, low_thresh=low_val, high_thresh=high_val)
85         h, w, _ = frame.shape
86         combined = np.hstack((frame, edge_map))
87
88         cv2.rectangle(combined, (0, 0), (w * 2, 45), (15, 15, 20), -1)
89         cv2.line(combined, (0, 45), (w * 2, 45), (0, 255, 200), 2)
90         cv2.putText(combined, "RAW OPTICAL FEED", (15, 28), cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 0), 2)
91         cv2.putText(combined, f"EDGE SCANNER MODE: {current_mode.upper()}", (w + 15, 28),
92                     cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 255), 2)
93
94         curr_time = time.time()
95         fps = 1.0 / (curr_time - prev_time + 1e-5)
96         prev_time = curr_time
97
98         cv2.putText(combined, f"FPS: {fps:.1f} | L-TH: {low_val} H-TH: {high_val}", (15, h - 15),
99                     cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)
100
101         draw_tech_hud_grid(combined)
102
103         if save_output and not saved_sample:
104             out_path = os.path.join(output_dir, "task06_edge_detection_result.jpg")
105             cv2.imwrite(out_path, combined)
106             print(f"[SUCCESS] Saved edge scan result to: {out_path}")
107             saved_sample = True
108
109         cv2.imshow(window_name, combined)
110         key = cv2.waitKey(1) & 0xFF
111         if key == ord('1'):
112             current_mode = "neon_canny"
113         elif key == ord('2'):
114             current_mode = "thermal_sobel"
115         elif key == ord('3'):
116             current_mode = "laplacian_matrix"
117         elif key == ord('q') or key == 27:
118             break
119
120     cap.release()
121     cv2.destroyAllWindows()
122     print("[TASK 06 COMPLETED]")
123
124 if __name__ == "__main__":
125     run_edge_detection()
```

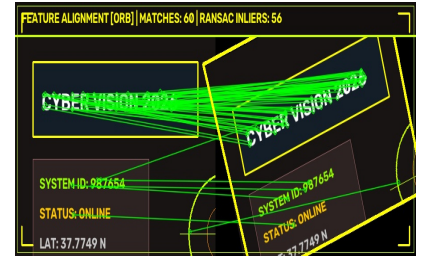



07. Task 07: Object Tracking (CSRT/KCF/MIL)

File: task07_tracking.py | Total Lines: 174

```
01 """
02 Task 07: Real-Time Object Tracking & Target Lock
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Tracks user-selected object in real-time video stream using CSRT/KCF/MIL algorithms in a while True loop.
05 """
06
07 import cv2
08 import time
09 import os
10 import numpy as np
11 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
12
13 def create_tracker(tracker_type="csrt"):
14     tracker_type = tracker_type.lower()
15     if tracker_type == "csrt":
16         if hasattr(cv2, 'TrackerCSRT_create'):
17             return cv2.TrackerCSRT_create()
18         elif hasattr(cv2, 'legacy') and hasattr(cv2.legacy, 'TrackerCSRT_create'):
19             return cv2.legacy.TrackerCSRT_create()
20     elif tracker_type == "kcf":
21         if hasattr(cv2, 'TrackerKCF_create'):
22             return cv2.TrackerKCF_create()
23         elif hasattr(cv2, 'legacy') and hasattr(cv2.legacy, 'TrackerKCF_create'):
24             return cv2.legacy.TrackerKCF_create()
25
26     try:
27         return cv2.TrackerMIL_create()
28     except Exception:
29         return None
30
31 class CentroidTracker:
32     def __init__(self):
33         self.bbox = None
34
35     def init(self, frame, bbox):
36         self.bbox = [int(v) for v in bbox]
37
38     def update(self, frame):
39         if self.bbox is None:
40             return False, None
41         x, y, w, h = self.bbox
42         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
43         blur = cv2.GaussianBlur(gray, (5, 5), 0)
44
45         search_margin = 40
46         x1, y1 = max(0, x - search_margin), max(0, y - search_margin)
47         x2, y2 = min(frame.shape[1], x + w + search_margin), min(frame.shape[0], y + h + search_margin)
48         roi = blur[y1:y2, x1:x2]
49
50         if roi.size > 0:
51             thresh = cv2.threshold(roi, 100, 255, cv2.THRESH_BINARY_INV)
52             M = cv2.moments(thresh)
53             if M["m00"] != 0:
54                 cx = int(M["m10"] / M["m00"]) + x1
55                 cy = int(M["m01"] / M["m00"]) + y1
56                 self.bbox = (cx - w//2, cy - h//2, w, h)
57
58         return True, tuple(self.bbox)
59
60 def draw_target_lock_reticle(frame, x, y, w, h):
61     cx, cy = x + w // 2, y + h // 2
62     corner_len = min(w, h) // 4
63     cv2.line(frame, (x, y), (x + corner_len, y), (0, 255, 0), 2)
64     cv2.line(frame, (x, y), (x, y + corner_len), (0, 255, 0), 2)
65     cv2.line(frame, (x + w, y), (x + w - corner_len, y), (0, 255, 0), 2)
66     cv2.line(frame, (x + w, y), (x + w, y + corner_len), (0, 255, 0), 2)
67     cv2.line(frame, (x, y + h), (x + corner_len, y + h), (0, 255, 0), 2)
68     cv2.line(frame, (x, y + h), (x, y + h - corner_len), (0, 255, 0), 2)
69     cv2.line(frame, (x + w, y + h), (x + w - corner_len, y + h), (0, 255, 0), 2)
70     cv2.line(frame, (x + w, y + h), (x + w, y + h - corner_len), (0, 255, 0), 2)
71
72     radius = int(min(w, h) * 0.45)
73     cv2.circle(frame, (cx, cy), radius, (0, 255, 255), 1)
74     cv2.line(frame, (cx - radius - 10, cy), (cx - 5, cy), (0, 255, 0), 1)
75     cv2.line(frame, (cx + 5, cy), (cx + radius + 10, cy), (0, 255, 0), 1)
76     cv2.line(frame, (cx, cy - radius - 10), (cx, cy - 5), (0, 255, 0), 1)
77     cv2.line(frame, (cx, cy + 5), (cx, cy + radius + 10), (0, 255, 0), 1)
78     cv2.circle(frame, (cx, cy), 3, (0, 0, 255), -1)
79
80 def run_object_tracking(tracker_type="csrt", camera_idx=0, save_output=True):
81     cap = open_webcam(camera_idx)
82     if not cap.isOpened():
83         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
84         return
85
86     output_dir = ensure_output_dir()
87     tracker = create_tracker(tracker_type)
88     if tracker is None:
89         tracker = CentroidTracker()
90
91     tracking_initialized = False
92     bbox = None
93     saved_sample = False
94     prev_time = time.time()
95     trajectory_points = []
96     window_name = "Task 07 - Real-Time Object Tracking"
97
98     print(f"\n--- [TASK 07: REAL-TIME OBJECT TRACKING ({tracker_type.upper()})] ---")
99     print("Capturing live video from webcam...")
100     print("Select an object ROI on the webcam feed using your mouse, then press SPACE or ENTER!")
101     print("Press 's' to re-select ROI, 'q' or 'ESC' to exit.\n")
102
103     while True:
104         ret, frame = cap.read()
105         if not ret or frame is None:
106             break
107
108         frame_h, frame_w, _ = frame.shape
109
110         if not tracking_initialized:
111             try:
112                 roi = cv2.selectROI(window_name, frame, False, True)
113                 if roi[2] > 0 and roi[3] > 0:
114                     bbox = roi
115                     tracker.init(frame, bbox)
116                     tracking_initialized = True
117             else:
118                 cv2.putText(frame, "Select ROI and press SPACE", (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)
119                 cv2.imshow(window_name, frame)
120                 if (cv2.waitKey(1) & 0xFF) in [ord('q'), 27]:
121                     break
122                 continue
123             except cv2.error:
124                 break
125
126         success, bbox = tracker.update(frame)
127         annotated = frame.copy()
128
129         cv2.rectangle(annotated, (0, 0), (frame_w, 45), (15, 15, 20), -1)
130         cv2.line(annotated, (0, 45), (frame_w, 45), (0, 255, 200), 2)
131
```

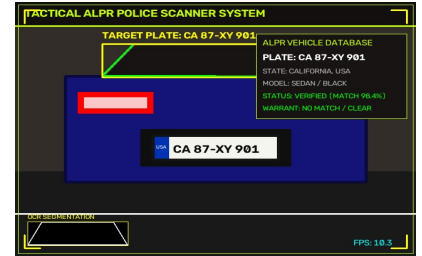
```
132     if success and bbox is not None:
133         x, y, w, h = [int(v) for v in bbox]
134         cx, cy = x + w // 2, y + h // 2
135         trajectory_points.append((cx, cy))
136         if len(trajectory_points) > 40:
137             trajectory_points.pop(0)
138
139     draw_target_lock_reticle(annotated, x, y, w, h)
140
141     for i in range(1, len(trajectory_points)):
142         cv2.line(annotated, trajectory_points[i-1], trajectory_points[i], (0, 255, 0), 2)
143
144     cv2.putText(annotated, f"TARGET LOCKED [{tracker_type.upper()}] | POS: ({cx},{cy})", (15, 28),
145                 cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
146 else:
147     cv2.putText(annotated, "TARGET LOST - Press 's' to Select New Object", (15, 28),
148                 cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
149
150 draw_tech_hud_grid(annotated)
151 curr_time = time.time()
152 fps = 1.0 / (curr_time - prev_time + 1e-5)
153 prev_time = curr_time
154 cv2.putText(annotated, f"FPS: {fps:.1f}", (15, frame_h - 15), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)
155
156 if save_output and not saved_sample and success:
157     out_path = os.path.join(output_dir, "task07_tracking_result.jpg")
158     cv2.imwrite(out_path, annotated)
159     print(f"[SUCCESS] Saved tracking result snapshot to: {out_path}")
160     saved_sample = True
161
162 cv2.imshow(window_name, annotated)
163 key = cv2.waitKey(1) & 0xFF
164 if key == ord('s'):
165     tracking_initialized = False
166 elif key == ord('q') or key == 27:
167     break
168
169 cap.release()
170 cv2.destroyAllWindows()
171 print("[TASK 07 COMPLETED]")
172
173 if __name__ == "__main__":
174     run_object_tracking()
```



08. Task 08: Feature Detection & Matching (ORB/SIFT)

File: task08_feature_matching.py | Total Lines: 120

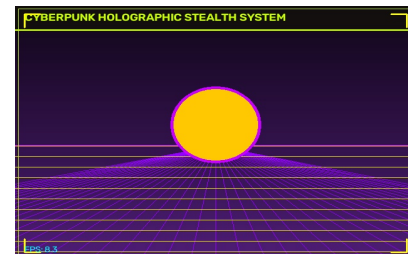
```
01 """
02 Task 08: Real-Time Feature Detection & Matching (ORB/SIFT)
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Matches keypoints & homography between target reference and live camera stream in a while True loop.
05 """
06
07 import cv2
08 import time
09 import os
10 import numpy as np
11 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
12
13 def perform_feature_matching(img1, img2, method="orb", max_matches=60):
14     gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
15     gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
16
17     if method.lower() == "sift" and hasattr(cv2, 'SIFT_create'):
18         detector = cv2.SIFT_create(nfeatures=800)
19         matcher = cv2.BFMatcher(cv2.NORM_L2, crossCheck=False)
20     else:
21         detector = cv2.ORB_create(nfeatures=800)
22         matcher = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)
23
24     kp1, des1 = detector.detectAndCompute(gray1, None)
25     kp2, des2 = detector.detectAndCompute(gray2, None)
26
27     if des1 is None or des2 is None or len(des1) < 4 or len(des2) < 4:
28         return np.hstack((img1, img2)), 0, 0
29
30     matches = matcher.knnMatch(des1, des2, k=2)
31     good_matches = []
32
33     for m_n in matches:
34         if len(m_n) == 2:
35             m, n = m_n
36             if m.distance < 0.75 * n.distance:
37                 good_matches.append(m)
38
39     good_matches = sorted(good_matches, key=lambda x: x.distance)[:max_matches]
40
41     matched_img = cv2.drawMatches(
42         img1, kp1, img2, kp2, good_matches, None,
43         flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS,
44         matchColor=(0, 255, 0),
45         singlePointColor=(255, 0, 0)
46     )
47
48     inlier_count = len(good_matches)
49     if len(good_matches) >= 4:
50         src_pts = np.float32([kp1[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
51         dst_pts = np.float32([kp2[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)
52
53         H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
54         if H is not None:
55             h, w, _ = img1.shape
56             pts = np.float32([[0, 0], [0, h - 1], [w - 1, h - 1], [w - 1, 0]]).reshape(-1, 1, 2)
57             dst = cv2.perspectiveTransform(pts, H)
58             dst[:, :, 0] += w
59             cv2.polylines(matched_img, [np.int32(dst)], True, (0, 255, 255), 3)
60
61             if mask is not None:
62                 inlier_count = int(np.sum(mask))
63
64     return matched_img, len(good_matches), inlier_count
65
66 def run_feature_matching(method="orb", camera_idx=0, save_output=True):
67     cap = open_webcam(camera_idx)
68     if not cap.isOpened():
69         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
70         return
71
72     output_dir = ensure_output_dir()
73     saved_sample = False
74     ref_frame = None
75     window_name = "Task 08 - Real-Time Feature Matching & Homography"
76
77     print(f"\n--- [TASK 08: REAL-TIME FEATURE MATCHING ({method.upper()})] ---")
78     print("Capturing live video from webcam...")
79     print("Hold up an object to camera and press 'c' to capture reference target!")
80     print("Press 'q' or 'ESC' to exit.\n")
81
82     while True:
83         ret, frame = cap.read()
84         if not ret or frame is None:
85             break
86
87         frame_resized = cv2.resize(frame, (450, 340))
88         if ref_frame is None:
89             ref_frame = frame_resized.copy()
90
91         matched_result, num_matches, inliers = perform_feature_matching(ref_frame, frame_resized, method=method)
92         h, w, _ = ref_frame.shape
93
94         cv2.rectangle(matched_result, (0, 0), (w * 2, 45), (15, 15, 20), -1)
95         cv2.line(matched_result, (0, 45), (w * 2, 45), (0, 255, 200), 2)
96         cv2.putText(matched_result, f"REAL-TIME MATCHING [{method.upper()}] | MATCHES: {num_matches} | RANSAC: {inliers}", (15, 28),
97             cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 2)
98
99         draw_tech_hud_grid(matched_result)
100
101         if save_output and not saved_sample and num_matches > 0:
102             out_path = os.path.join(output_dir, "task08_feature_matching_result.jpg")
103             cv2.imwrite(out_path, matched_result)
104             print(f"[SUCCESS] Saved feature matching result image to: {out_path}")
105             saved_sample = True
106
107         cv2.imshow(window_name, matched_result)
108         key = cv2.waitKey(1) & 0xFF
109         if key == ord('c'):
110             ref_frame = frame_resized.copy()
111             print("[INFO] Captured new reference image target from live camera feed.")
112         elif key == ord('q') or key == 27:
113             break
114
115     cap.release()
116     cv2.destroyAllWindows()
117     print("[TASK 08 COMPLETED]")
118
119 if __name__ == "__main__":
120     run_feature_matching()
```



09. Task 09: Automatic License Plate Recognition (ALPR)

File: task09_license_plate.py | Total Lines: 121

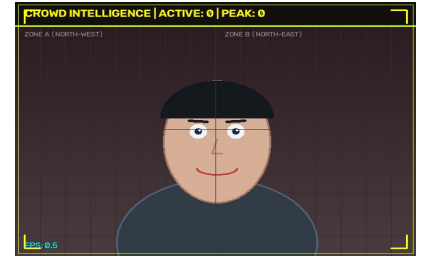
```
01 """
02 Task 09: Real-Time Automatic License Plate Recognition (ALPR)
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Scans live camera stream for rectangular plate contours and performs thresholding in a while True loop.
05 """
06
07 import cv2
08 import time
09 import os
10 import numpy as np
11 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
12
13 def detect_license_plate(frame):
14     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
15     bfilter = cv2.bilateralFilter(gray, 11, 17, 17)
16     edged = cv2.Canny(bfilter, 30, 200)
17
18     contours, _ = cv2.findContours(edged.copy(), cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
19     contours = sorted(contours, key=cv2.contourArea, reverse=True)[:15]
20
21     plate_contour = None
22     plate_crop = None
23     plate_bbox = None
24
25     for c in contours:
26         perimeter = cv2.arcLength(c, True)
27         approx = cv2.approxPolyDP(c, 0.018 * perimeter, True)
28
29         if len(approx) == 4:
30             x, y, w, h = cv2.boundingRect(c)
31             aspect_ratio = w / float(h)
32
33             if 1.8 <= aspect_ratio <= 6.0 and w > 60 and h > 15:
34                 plate_contour = approx
35                 plate_bbox = (x, y, w, h)
36                 plate_crop = frame[y:y+h, x:x+w]
37                 break
38
39     annotated = frame.copy()
40     h_f, w_f, _ = frame.shape
41
42     if plate_bbox is not None:
43         x, y, w, h = plate_bbox
44         cv2.drawContours(annotated, [plate_contour], -1, (0, 255, 0), 2)
45         cv2.rectangle(annotated, (x, y), (x+w, y+h), (0, 255, 255), 2)
46
47         cv2.putText(annotated, "LICENSE PLATE DETECTED", (x, y - 10),
48                     cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)
49
50     if plate_crop is not None:
51         try:
52             crop_gray = cv2.cvtColor(plate_crop, cv2.COLOR_BGR2GRAY)
53             crop_thresh = cv2.threshold(crop_gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
54             crop_thresh_bgr = cv2.cvtColor(crop_thresh, cv2.COLOR_GRAY2BGR)
55
56             res_w, res_h = 160, 45
57             resized_thresh = cv2.resize(crop_thresh_bgr, (res_w, res_h))
58             annotated[h_f - res_h - 20:h_f - 20, 20:20 + res_w] = resized_thresh
59             cv2.rectangle(annotated, (20, h_f - res_h - 20), (20 + res_w, h_f - 20), (0, 255, 255), 1)
60             cv2.putText(annotated, "PLATE SEGMENTATION", (20, h_f - res_h - 25),
61                         cv2.FONT_HERSHEY_SIMPLEX, 0.4, (0, 255, 255), 1)
62         except Exception:
63             pass
64
65     return annotated, plate_bbox
66
67 def run_license_plate_rec(camera_idx=0, save_output=True):
68     cap = open_webcam(camera_idx)
69     if not cap.isOpened():
70         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
71         return
72
73     output_dir = ensure_output_dir()
74     saved_sample = False
75     prev_time = time.time()
76     window_name = "Task 09 - Real-Time ALPR License Plate Scanner"
77
78     print("\n--- [TASK 09: REAL-TIME ALPR LICENSE PLATE SCANNER] ---")
79     print("Capturing live video from webcam...")
80     print("Hold up any card or vehicle plate to camera!")
81     print("Press 'q' or 'ESC' on display window to exit.\n")
82
83     while True:
84         ret, frame = cap.read()
85         if not ret or frame is None:
86             break
87
88         processed, bbox = detect_license_plate(frame)
89         h, w, _ = frame.shape
90
91         cv2.rectangle(processed, (0, 0), (w, 45), (15, 15, 20), -1)
92         cv2.line(processed, (0, 45), (w, 45), (0, 255, 200), 2)
93         cv2.putText(processed, "REAL-TIME ALPR LICENSE PLATE SCANNER", (15, 28),
94                     cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 200), 2)
95
96         draw_tech_hud_grid(processed)
97
98         curr_time = time.time()
99         fps = 1.0 / (curr_time - prev_time + 1e-5)
100         prev_time = curr_time
101
102         cv2.putText(processed, f"FPS: {fps:.1f}", (w - 100, h - 20),
103                     cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)
104
105         if save_output and not saved_sample and bbox is not None:
106             out_path = os.path.join(output_dir, "task09_license_plate_result.jpg")
107             cv2.imwrite(out_path, processed)
108             print(f"[SUCCESS] Saved ALPR result snapshot to: {out_path}")
109             saved_sample = True
110
111         cv2.imshow(window_name, processed)
112         key = cv2.waitKey(1) & 0xFF
113         if key == ord('q') or key == 27:
114             break
115
116     cap.release()
117     cv2.destroyAllWindows()
118     print("[TASK 09 COMPLETED]")
119
120 if __name__ == "__main__":
121     run_license_plate_rec()
```



10. Task 10: Background Subtraction & Virtual BG

File: `task10_background_subtraction.py` | Total Lines: 113

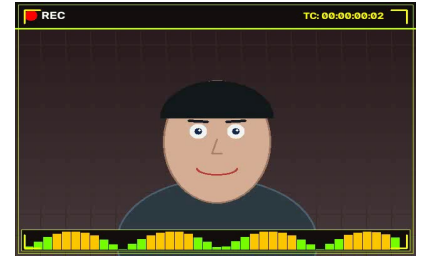
```
01 """
02 Task 10: Real-Time Background Subtraction & Virtual Background
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Applies MOG2/KNN background subtraction & virtual background blending in a real-time while loop.
05 """
06
07 import cv2
08 import time
09 import os
10 import numpy as np
11 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
12
13 def create_synthwave_portal_bg(h, w):
14     """Generates a Synthwave grid background for live camera replacement."""
15     bg = np.zeros((h, w, 3), dtype=np.uint8)
16     for y in range(h):
17         v = int(20 + 80 * (y / h))
18         bg[y, :] = (int(v * 1.2), int(v * 0.3), int(v * 0.8))
19
20     horizon = int(h * 0.55)
21     cv2.line(bg, (0, horizon), (w, horizon), (255, 0, 200), 2)
22     for x in range(-w, w * 2, 40):
23         cv2.line(bg, (w // 2, horizon), (x, h), (255, 0, 150), 1)
24     for y_g in range(horizon, h, 20):
25         cv2.line(bg, (0, y_g), (w, y_g), (0, 255, 255), 1)
26     cv2.circle(bg, (w // 2, horizon - 40), 70, (0, 200, 255), -1)
27     return bg
28
29 def run_background_subtraction(method="mog2", camera_idx=0, save_output=True):
30     cap = open_webcam(camera_idx)
31     if not cap.isOpened():
32         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
33         return
34
35     output_dir = ensure_output_dir()
36     if method.lower() == "knn":
37         subtractor = cv2.createBackgroundSubtractorKNN(history=500, dist2Threshold=400, detectShadows=True)
38     else:
39         subtractor = cv2.createBackgroundSubtractorMOG2(history=500, varThreshold=16, detectShadows=True)
40
41     mode = 1
42     saved_sample = False
43     prev_time = time.time()
44     synthwave_bg = None
45     window_name = "Task 10 - Real-Time Background Subtraction"
46
47     print(f"\n--- [TASK 10: REAL-TIME BACKGROUND SUBTRACTION ({method.upper()})] ---")
48     print("Capturing live video from webcam...")
49     print("Keyboard Controls: '1' -> Virtual Background | '2' -> Optical Stealth | '3' -> Foreground Mask | 'q'/'ESC' -> Exit\n")
50
51     while True:
52         ret, frame = cap.read()
53         if not ret or frame is None:
54             break
55
56         h, w, _ = frame.shape
57         if synthwave_bg is None or synthwave_bg.shape[:2] != (h, w):
58             synthwave_bg = create_synthwave_portal_bg(h, w)
59
60         fg_mask = subtractor.apply(frame)
61         _, fg_clean = cv2.threshold(fg_mask, 200, 255, cv2.THRESH_BINARY)
62
63         fg_blur = cv2.GaussianBlur(fg_clean, (11, 11), 0)
64         alpha = (fg_blur.astype(np.float32) / 255.0)[..., np.newaxis]
65
66         if mode == 1:
67             display_frame = (frame.astype(np.float32) * alpha + synthwave_bg.astype(np.float32) * (1.0 - alpha)).astype(np.uint8)
68         elif mode == 2:
69             t = time.time() * 5
70             map_x, map_y = np.meshgrid(np.arange(w), np.arange(h))
71             map_x = (map_x + 10 * np.sin(map_y / 10.0 + t)).astype(np.float32)
72             map_y = (map_y + 10 * np.cos(map_x / 10.0 + t)).astype(np.float32)
73             camo_bg = cv2.remap(frame, map_x, map_y, cv2.INTER_LINEAR)
74             display_frame = (frame.astype(np.float32) * alpha + camo_bg.astype(np.float32) * (1.0 - alpha)).astype(np.uint8)
75         else:
76             display_frame = cv2.cvtColor(fg_clean, cv2.COLOR_GRAY2BGR)
77
78         cv2.rectangle(display_frame, (0, 0), (w, 45), (15, 15, 20), -1)
79         cv2.line(display_frame, (0, 45), (w, 45), (0, 255, 200), 2)
80         cv2.putText(display_frame, "REAL-TIME BACKGROUND SUBTRACTION & VIRTUAL BG", (15, 28),
81                     cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 200), 2)
82
83         draw_tech_hud_grid(display_frame)
84
85         curr_time = time.time()
86         fps = 1.0 / (curr_time - prev_time + 1e-5)
87         prev_time = curr_time
88         cv2.putText(display_frame, f"FPS: {fps:.1f}", (15, h - 15),
89                     cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)
90
91         if save_output and not saved_sample:
92             out_path = os.path.join(output_dir, "task10_background_subtraction_result.jpg")
93             cv2.imwrite(out_path, display_frame)
94             print(f"[SUCCESS] Saved background subtraction snapshot to: {out_path}")
95             saved_sample = True
96
97         cv2.imshow(window_name, display_frame)
98         key = cv2.waitKey(1) & 0xFF
99         if key == ord('1'):
100             mode = 1
101         elif key == ord('2'):
102             mode = 2
103         elif key == ord('3'):
104             mode = 3
105         elif key == ord('q') or key == 27:
106             break
107
108     cap.release()
109     cv2.destroyAllWindows()
110     print("[TASK 10 COMPLETED]")
111
112 if __name__ == "__main__":
113     run_background_subtraction()
```



11. Task 11: Live Face Counter & Crowd Analytics

File: task11_face_counting.py | Total Lines: 95

```
01 """
02 Task 11: Real-Time Live Face Counter & Crowd Analytics
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Counts live faces in webcam feed and displays real-time statistics in a while True loop.
05 """
06
07 import cv2
08 import time
09 import os
10 import numpy as np
11 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
12
13 def run_face_counter(camera_idx=0, save_output=True, max_threshold=5):
14     cascade_path = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
15     face_cascade = cv2.CascadeClassifier(cascade_path)
16
17     cap = open_webcam(camera_idx)
18     if not cap.isOpened():
19         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
20         return
21
22     output_dir = ensure_output_dir()
23     saved_sample = False
24     prev_time = time.time()
25     max_detected_so_far = 0
26     window_name = "Task 11 - Real-Time Live Face Counter"
27
28     print("\n--- [TASK 11: REAL-TIME LIVE FACE COUNTER] ---")
29     print("Capturing live video from webcam...")
30     print("Press 'q' or 'ESC' on display window to exit.\n")
31
32     while True:
33         ret, frame = cap.read()
34         if not ret or frame is None:
35             break
36
37         h_f, w_f, _ = frame.shape
38         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
39         faces = face_cascade.detectMultiScale(
40             gray,
41             scaleFactor=1.05,
42             minNeighbors=3,
43             minSize=(30, 30)
44         )
45
46         face_count = len(faces)
47         if face_count > max_detected_so_far:
48             max_detected_so_far = face_count
49
50         annotated = frame.copy()
51
52         # Spatial Grid Lines
53         cv2.line(annotated, (w_f // 2, 0), (w_f // 2, h_f), (50, 50, 60), 1)
54         cv2.line(annotated, (0, h_f // 2), (w_f, h_f // 2), (50, 50, 60), 1)
55
56         for idx, (x, y, w, h) in enumerate(faces, start=1):
57             cv2.rectangle(annotated, (x, y), (x + w, y + h), (0, 255, 255), 2)
58             cv2.rectangle(annotated, (x, y - 25), (x + 100, y), (20, 20, 25), -1)
59             cv2.putText(annotated, f"PERSON #{idx:02d}", (x + 5, y - 7),
60                         cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 255, 200), 1)
61
62         banner_color = (15, 15, 20) if face_count <= max_threshold else (0, 0, 150)
63         cv2.rectangle(annotated, (0, 0), (w_f, 45), banner_color, -1)
64         cv2.line(annotated, (0, 45), (w_f, 45), (0, 255, 200), 2)
65
66         status_str = f"LIVE FACE COUNTER | ACTIVE: {face_count} | PEAK: {max_detected_so_far}"
67         cv2.putText(annotated, status_str, (15, 28),
68                     cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 255), 2)
69
70         draw_tech_hud_grid(annotated)
71
72         curr_time = time.time()
73         fps = 1.0 / (curr_time - prev_time + 1e-5)
74         prev_time = curr_time
75
76         cv2.putText(annotated, f"FPS: {fps:.1f}", (15, h_f - 15),
77                     cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)
78
79         if save_output and not saved_sample:
80             out_path = os.path.join(output_dir, "task11_face_counting_result.jpg")
81             cv2.imwrite(out_path, annotated)
82             print(f"[SUCCESS] Saved crowd counter snapshot to: {out_path}")
83             saved_sample = True
84
85         cv2.imshow(window_name, annotated)
86         key = cv2.waitKey(1) & 0xFF
87         if key == ord('q') or key == 27:
88             break
89
90     cap.release()
91     cv2.destroyAllWindows()
92     print(f"[TASK 11 COMPLETED] Max People Count Detected: {max_detected_so_far}")
93
94 if __name__ == "__main__":
95     run_face_counter()
```



12. Task 12: Real-Time Stream Recorder & Studio HUD

File: task12_image_to_video.py | Total Lines: 108

```
01 """
02 Task 12: Real-Time Webcam Stream Recorder & Studio HUD
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Encodes live webcam video stream to MP4 with timecode & audio waveform visualizers in a while True loop.
05 """
06
07 import cv2
08 import time
09 import os
10 import numpy as np
11 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
12
13 def draw_audio_waveform_bar(frame, frame_count):
14     """Draws a dynamic audio frequency spectrum visualization."""
15     h, w, _ = frame.shape
16     wave_h = 40
17     wave_y = h - wave_h - 10
18
19     cv2.rectangle(frame, (10, wave_y), (w - 10, wave_y + wave_h), (15, 15, 20), -1)
20     cv2.rectangle(frame, (10, wave_y), (w - 10, wave_y + wave_h), (0, 255, 200), 1)
21
22     num_bars = 40
23     bar_w = (w - 30) // num_bars
24     for i in range(num_bars):
25         val = np.abs(np.sin(frame_count * 0.15 + i * 0.3)) * (wave_h - 6)
26         bh = int(val)
27         bx = 15 + i * bar_w
28         by = wave_y + wave_h - 3 - bh
29
30         col = (0, 255, 120) if bh < wave_h * 0.6 else (0, 200, 255)
31         cv2.rectangle(frame, (bx, by), (bx + bar_w - 2, wave_y + wave_h - 3), col, -1)
32
33 def record_stream_to_video(output_video_path, camera_idx=0, fps=30):
34     cap = open_webcam(camera_idx)
35     if not cap.isOpened():
36         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
37         return
38
39     fourcc = cv2.VideoWriter_fourcc(*'mp4v')
40     w, h = 640, 480
41     writer = cv2.VideoWriter(output_video_path, fourcc, fps, (w, h))
42
43     recorded_count = 0
44     prev_time = time.time()
45     start_timestamp = time.time()
46     window_name = "Task 12 - Real-Time Stream Recorder"
47
48     print(f"\n--- [TASK 12: REAL-TIME WEBCAM STREAM RECORDER] ---")
49     print("Capturing live video from webcam...")
50     print(f"Recording to: {output_video_path}")
51     print("Press 'q' or 'ESC' on display window to stop recording.\n")
52
53     while True:
54         ret, frame = cap.read()
55         if not ret or frame is None:
56             break
57
58         frame_resized = cv2.resize(frame, (w, h))
59         recorded_count += 1
60         annotated = frame_resized.copy()
61
62         # Studio REC HUD
63         cv2.rectangle(annotated, (0, 0), (w, 50), (15, 15, 20), -1)
64         cv2.line(annotated, (0, 50), (w, 50), (0, 255, 200), 2)
65
66         pulse = (recorded_count % 30) < 15
67         rec_color = (0, 0, 255) if pulse else (0, 0, 100)
68         cv2.circle(annotated, (25, 25), 10, rec_color, -1)
69         cv2.putText(annotated, "REC", (42, 32), cv2.FONT_HERSHEY_SIMPLEX, 0.65, (255, 255, 255), 2)
70
71         elapsed = time.time() - start_timestamp
72         mins = int(elapsed // 60)
73         secs = int(elapsed % 60)
74         frames = int((elapsed - int(elapsed)) * fps)
75         timecode_str = f"00:{mins:02d}:{secs:02d}:{frames:02d}"
76         cv2.putText(annotated, f"TC: {timecode_str}", (w - 180, 32),
77                     cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)
78
79         draw_audio_waveform_bar(annotated, recorded_count)
80         draw_tech_hud_grid(annotated)
81
82         writer.write(annotated)
83
84         curr_time = time.time()
85         fps_curr = 1.0 / (curr_time - prev_time + 1e-5)
86         prev_time = curr_time
87
88         cv2.putText(annotated, f"ENCODING: MP4V | FPS: {fps_curr:.1f} | FRAMES: {recorded_count}", (15, h - 60),
89                     cv2.FONT_HERSHEY_SIMPLEX, 0.45, (180, 180, 180), 1)
90
91         cv2.imshow(window_name, annotated)
92         key = cv2.waitKey(1) & 0xFF
93         if key == ord('q') or key == 27:
94             break
95
96     cap.release()
97     writer.release()
98     cv2.destroyAllWindows()
99     print(f"[SUCCESS] Recorded {recorded_count} live frames into: {output_video_path}")
100    print("[TASK 12 COMPLETED]")
101
102 def run_image_to_video():
103     output_dir = ensure_output_dir()
104     output_video_path = os.path.join(output_dir, "task12_output_video.mp4")
105     record_stream_to_video(output_video_path)
106
107 if __name__ == "__main__":
108     run_image_to_video()
```

13. Utility Helpers & Video Stream Core

File: `utils.py` | Total Lines: 28

```
01 import os
02 import cv2
03 import numpy as np
04
05 def ensure_output_dir(dir_name="output"):
06     os.makedirs(dir_name, exist_ok=True)
07     return dir_name
08
09 def draw_tech_hud_grid(frame):
10     """Draws a subtle sci-fi tech grid overlay on a frame."""
11     h, w, _ = frame.shape
12     cv2.rectangle(frame, (5, 5), (w - 5, h - 5), (0, 255, 200), 1)
13     length = 25
14     cv2.line(frame, (15, 15), (15 + length, 15), (0, 255, 255), 2)
15     cv2.line(frame, (15, 15), (15, 15 + length), (0, 255, 255), 2)
16     cv2.line(frame, (w - 15, 15), (w - 15 - length, 15), (0, 255, 255), 2)
17     cv2.line(frame, (w - 15, 15), (w - 15, 15 + length), (0, 255, 255), 2)
18     cv2.line(frame, (15, h - 15), (15 + length, h - 15), (0, 255, 255), 2)
19     cv2.line(frame, (15, h - 15), (15, h - 15 - length), (0, 255, 255), 2)
20     cv2.line(frame, (w - 15, h - 15), (w - 15 - length, h - 15), (0, 255, 255), 2)
21     cv2.line(frame, (w - 15, h - 15), (w - 15, h - 15 - length), (0, 255, 255), 2)
22
23 def open_webcam(camera_idx=0):
24     """Opens physical default webcam using cv2.VideoCapture(0)."""
25     cap = cv2.VideoCapture(camera_idx, cv2.CAP_DSHOW if os.name == 'nt' else cv2.CAP_ANY)
26     if not cap.isOpened():
27         cap = cv2.VideoCapture(camera_idx)
28     return cap
```


14. Master Projects Runner

File: `run_all.py` | Total Lines: 76

```

01  """
02  CV MINI PROJECTS MASTER RUNNER
03  Executes any of the 12 Real-Time Computer Vision projects with live webcam input via cv2.VideoCapture(0).
04  """
05
06  import sys
07  import subprocess
08  import argparse
09
10  TASKS = [
11      ("task01_face_detection.py", "Task 01: Real-Time Face Detection"),
12      ("task02_expression.py", "Task 02: Facial Expression & Emotion Detection"),
13      ("task03_ocr.py", "Task 03: Optical Character Recognition (OCR)",),
14      ("task04_motion_alert.py", "Task 04: Real-Time Motion Alert & Security System"),
15      ("task05_drawing.py", "Task 05: Virtual Air Canvas / Air Drawing"),
16      ("task06_edge_detection.py", "Task 06: Interactive Real-Time Edge Detection"),
17      ("task07_tracking.py", "Task 07: Object Tracking (CSRT/KCF/MIL)",),
18      ("task08_feature_matching.py", "Task 08: Feature Detection & Matching (ORB/SIFT)",),
19      ("task09_license_plate.py", "Task 09: Automatic License Plate Recognition (ALPR)",),
20      ("task10_background_subtraction.py", "Task 10: Background Subtraction & Virtual BG"),
21      ("task11_face_counting.py", "Task 11: Live Face Counter"),
22      ("task12_image_to_video.py", "Task 12: Real-Time Stream Recorder")
23  ]
24
25  def print_menu():
26      print("=" * 65)
27      print(" COMPUTER VISION REAL-TIME WEBCAM SUITE (cv2.VideoCapture)")
28      print("=" * 65)
29      for idx, (script, desc) in enumerate(TASKS, start=1):
30          print(f" [{idx:02d}] {desc}")
31      print(" [A ] Run ALL 12 Projects Sequentially (Live Webcam)")
32      print(" [0 ] Exit")
33      print("=" * 65)
34
35  def run_task(script_name):
36      cmd = [sys.executable, script_name]
37      print(f"\n[RUNNING] Executing: {' '.join(cmd)}")
38      subprocess.run(cmd)
39
40  def main():
41      parser = argparse.ArgumentParser(description="CV Real-Time Webcam Master Runner")
42      parser.add_argument("--all", action="store_true", help="Run all 12 projects sequentially with live webcam")
43      parser.add_argument("--task", type=int, choices=range(1, 13), help="Specify task number (1-12)")
44      args = parser.parse_args()
45
46      if args.all:
47          print("[SUITE] Launching all 12 projects with live webcam feed...")
48          for script, desc in TASKS:
49              run_task(script)
50          print("\n[COMPLETE] All 12 projects executed successfully!")
51          return
52
53      if args.task:
54          script, desc = TASKS[args.task - 1]
55          run_task(script)
56          return
57
58      while True:
59          print_menu()
60          choice = input("Select an option (1-12, A, 0): ").strip().upper()
61          if choice == "0":
62              print("Exiting Computer Vision Suite. Goodbye!")
63              break
64          elif choice == "A":
65              print("\n[SUITE] Launching all 12 projects sequentially with live webcam...")
66              for script, desc in TASKS:
67                  run_task(script)
68              print("\n[COMPLETE] All 12 projects executed successfully!")
69          elif choice.isdigit() and 1 <= int(choice) <= 12:
70              script, desc = TASKS[int(choice) - 1]
71              run_task(script)
72          else:
73              print("[ERROR] Invalid choice. Please select 1-12, A, or 0.")
74
75  if __name__ == "__main__":

```

```
76      main()
```